The Kerekes Handshake™

Universal Verification Protocol
Whitepaper & Full Specification v1.6

Author: Jeffrey Kerekes — Systems Practitioner

Why I Built This

I hate writing resumes. After 15+ years self-employed, I realized that modern AI resume tools force everything into a fluffy two-page limit, and every bullet still feels like *"tell, don't show."*

I created the Kerekes Handshake: a dead-simple way to link every claim on your resume to real evidence. Now recruiters or AI can audit you in one click instead of taking your word for it. This won't replace digital wallets for driver's licenses and diplomas — but what about the things no third party would ever certify? Solo audits. Civic interventions. DIY builds. Local wins. That's the gap this fills.

> *Demo note: This specification uses live proof from a real career. For privacy when you copy it, redact sensitive details in your own version. Real data beats synthetic every time.*

Abstract

Traditional trust: Claim → Narrative → Guess

Kerekes Handshake: Claim → Evidence → Verification

Vault Resume (v1.6): Claim stub → AI audit → Evidence vault → Verdict

The Kerekes Handshake™ is a universal protocol for moving any domain from **Probabilistic Trust** (guessing based on narrative) to **Deterministic Verification** (auditing based on evidence). The same architecture that verifies a resume also verifies a plumber's license, a property's condition history, a nonprofit's financials, or a politician's 30-year voting record. v1.6 adds the Vault Resume and the Universal Claims Standard.

1. Core Concept

Every bullet on a traditional resume is an unverified assertion. The Kerekes Handshake replaces the implicit trust chain with an explicit evidence chain. The protocol is not a credential issuer — it is a

structured way to surface evidence you already have.

- No blockchain required.
- No third-party issuer required.
- No wallet required — yet. (See Section 9 on future-proofing.)

2. System Architecture — The Three-Layer Model

A compliant Handshake implementation operates across three layers:

- **Layer 1 — Narrative (Resume):** The human-readable interface. Professional claims are tagged with `data-kcm` attributes.
- **Layer 2 — Registry (Index):** The `claims.json` ledger. Maps each claim ID to one or more absolute evidence URIs.
- **Layer 3 — Integrity (The Seal):** The `site_manifest.json.asc` — a PGP-signed manifest containing SHA-256 hashes of all artifacts. The cryptographic Root of Trust.

```
Resume (data-kcm) → claims.json → site_manifest.json.asc → /evidence/
```

AI agent flow: read resume → detect KCM tags → load registry → verify hashes → retrieve artifacts → return verdict.

3. File Structure

Place these files at your website root:

```
/index.html               <- The Resume (tagged with data-kcm)
/claims.json              <- The Map (claim IDs -> evidence URIs)
/site_manifest.json.asc   <- The PGP-Signed Root of Trust
/query/                   <- The AI Audit Portal
/evidence/                <- The Vault: PDFs + matching .txt sidecars
/openapi.yaml             <- Machine-readable API instructions
/llms-full.txt            <- High-density LLM pre-flight context
/.well-known/ai-plugin.json  <- Agent discovery manifest
```

4. Step-by-Step Implementation

This section is written for non-developers. You do not need to write code to implement the core protocol.

Step 1 — Collect & Upload Your Evidence

Create a folder called `/evidence/` on your website. Upload your real primary-source documents: inspection reports, FOI rulings, audit spreadsheets, certificates, anything that verifies a claim you make.

Step 2 — Generate the Text Bridge (v1.5 Requirement)

> **Why this matters:** Most AI crawlers cannot reliably parse binary PDF streams in real-time. They stall, skip, or hallucinate. The Text Bridge solves this.

For every PDF in `/evidence/`, create a matching `.txt` sidecar with the identical filename:

```
budget_audit_2011.pdf
budget_audit_2011.txt   <- same name, .txt extension
```

Automatic generation (requires `poppler-utils`):

```
for f in *.pdf; do pdftotext "$f" "${f%.pdf}.txt"; done
```

Step 3 — Create claims.json

Create a file named `claims.json` at your site root. This is the map connecting each resume claim to its evidence files.

```
[
  {
    "claim_id": "budget_audit_2011",
    "title": "Municipal Budget Reconstruction",
    "description": "Reconstructed city budget using raw FOI data.",
    "domain": "Civic Systems",
    "evidence": [
      "https://yourdomain.com/evidence/budget_audit_2011.pdf",
      "https://yourdomain.com/evidence/budget_audit_2011.txt"
    ],
    "outcome": "Identified $38k/household hidden debt liability"
  }
]
```

Step 4 — Sign Your Manifest (PGP Integrity Seal)

Generate SHA-256 hashes for all evidence files, store them in `site_manifest.json`, then sign it:

```
gpg --clearsign site_manifest.json
# Produces: site_manifest.json.asc
```

Step 5 — Tag Your Resume with KCM

Wrap each professional claim in a `data-kcm` tag:

```
<article data-kcm="budget_audit_2011"
         aria-label="KCM Claim: budget_audit_2011">
  <h3>Municipal Budget Reconstruction</h3>
  <p>Reconstructed municipal budget using raw data.</p>
  <footer class="evidence-grid">
```

```
    <a href="/evidence/budget_audit_2011.pdf">Audit [.pdf]</a>
    <a href="/evidence/budget_audit_2011.txt">Text Bridge [.txt]</a>
  </footer>
</article>
```

Step 6 — Add the Audit Link to Your Resume

Add this line to the top of your resume:

```
Audit this resume with AI ->  https://yourdomain.com/query
```

5. KCM — Kerekes Claim Markup

KCM is a lightweight semantic HTML micro-layer that tethers narrative bullets to machine-readable evidence. It requires no JavaScript and no external library.

Required Elements

- `data-kcm="[claim_id]"` — Placed on the parent `<article>` of a claim. Must match a `claim_id` in `claims.json`.

Optional Enhancement Attributes

- `class="ai-hook"` — Marks specific keywords, case numbers, or forensic pivots within claim text.
- `itemprop="evidenceOrigin"` — Applied to evidence links for Schema.org structured data parsers.

6. claims.json — Field Specification

Field	Required	Description
claim_id	✓	Unique slug matching the data-kcm value
title	✓	Short human-readable label for the claim
evidence	✓	Array of absolute URIs — include both .pdf and .txt versions
outcome	✓	The verifiable result of the intervention
domain		System category: Human, Civic, Physical, or Ecological
description		Narrative summary of the diagnosis and intervention

7. AI Integration

AI systems with web access can autonomously traverse the full audit chain:

- **Step 1:** Read `llms-full.txt` for pre-flight context.
- **Step 2:** Fetch resume HTML — read raw source to detect `data-kcm` attributes.
- **Step 3:** Fetch `claims.json` — map claim IDs to evidence URIs.
- **Step 4:** Retrieve artifacts — use `.txt` sidecar if PDF fails.
- **Step 5:** Verify SHA-256 hashes against `site_manifest.json.asc`.
- **Step 6:** Cross-corroborate with `/archive/` press records.
- **Step 7:** Return verdict: VERIFIED / CONFLICT / UNSTABLE per claim.

8. Benefits

- **Verifiable credibility:** Claims are backed by primary artifacts, not self-assertion.
- **AI-readable by design:** Built for the era of AI-assisted hiring.
- **Transparency:** Evidence is publicly accessible with open CORS.
- **Portable:** Your evidence lives on your own domain.
- **Self-sovereign:** No issuer needed. You hold and serve your own proof.
- **Keyword stuffing dies:** Without vault evidence, inflated claims are instantly exposed.
- **Future-compatible:** Evidence hashes can be exported into verifiable credentials.

9. The Vault Resume — Presentation Layer Standard (v1.6)

The core insight of v1.6: Stop explaining your work in prose. Put the wins on one dense half-page. Let AI pull the vault.

Philosophy

The Vault Resume resolves the core tension in modern hiring: recruiters scan in six seconds, but AI can audit in sixty. Traditional resumes try to serve both audiences with prose — and fail at both. The Vault Resume separates them cleanly:

- **Human layer (half-page):** Dense claim stubs that telegraph wins without explanation. Recruiters see enough to think: *"worth a query."*
- **AI layer (infinite vault):** Evidence PDFs, audits, press records, government filings — everything needed to verify, contextualize, and assess significance.
- **Evidence is the explanation:** No adjectives. No paragraphs. The vault speaks instead.
- **Keyword stuffing dies:** A claim without vault evidence is instantly flagged. Real builders win; resume inflation loses.

```
HUMAN LAYER (half-page):   Dense stubs -> six-second scan -> "worth a query"
                                          |
                                          v
AI LAYER (infinite vault):  Fetch evidence -> verify claims -> return verdict
```

Stub Format Specification

Each claim on the visible resume is a single dense line:

```
[Domain]: [role/intervention], [key metric], [date range] -- REF: [claim_id]
```

Examples from the reference implementation:

```
Civic: Municipal budget audit, $38k/household debt exposed,
       FIC 2009-014 secured, 2007-2011 -- REF: budget_audit_2007_2011

Clinical: ER throughput rebuild, 44->79 hospital network,
          divert status eliminated, 2001-2003 -- REF: clinical_throughput_cambridge

Physical: Solo residential build x2, structural masonry + MEP,
          CO achieved, 2008 -- REF: asset_lifecycle_physical
```

Stub Rules

- One line per claim — no paragraph prose
- Lead with the domain, follow immediately with the metric or outcome
- Include the date range

- Always end with `REF: [claim_id]` — AI agents use this to locate the vault entry instantly
- No adjectives — "successfully," "expertly," "significantly" are banned; the evidence qualifies the claim

KCM Implementation for Vault Resume

The stub text is wrapped in a standard KCM article. Visible content is the stub; evidence links live in the footer:

```
<article class="claim" data-kcm="budget_audit_2007_2011"
         aria-label="KCM Claim: budget_audit_2007_2011"
         itemscope itemtype="https://schema.org/CreativeWork">
  <p itemprop="description">
    Civic: Municipal budget audit,
    <span class="ai-hook" title="Per-household debt">$38k/household debt exposed</span>,
    <span class="ai-hook" title="FOI Case: FIC 2009-014">FIC 2009-014 secured</span>,
    2007-2011
  </p>
  <footer class="evidence-grid">
    <a href="/evidence/2011-main-spreadsheet-all.pdf">Audit [.pdf]</a>
    <a href="/evidence/2011-main-spreadsheet-all.txt">Text Bridge [.txt]</a>
  </footer>
</article>
```

What Belongs in the Vault vs. On the Page

On the page (human layer)	In the vault (AI layer)
Claim stub — one line	Full evidence PDFs + .txt sidecars
Key metric or outcome	Press coverage and archive records
Date range	Inspection reports, rulings, filings
REF: claim_id	Certifications and government documents
Audit link at top	PGP-signed manifest

v1.6 Compliance Note

v1.6 compliance requires v1.5 compliance as its foundation. The Vault Resume standard applies only to the Narrative Layer presentation format — all Registry, Integrity, and Text Bridge requirements from v1.5 remain unchanged. A v1.5 implementation can be upgraded to v1.6 by reformatting the resume HTML to use dense stubs. **No infrastructure changes are required.**

10. Future-Proofing & Hybrid Vision

The Kerekes Handshake is an *interim bridge* — designed for the class of accomplishments no institution will ever certify.

Once digital wallets and Verifiable Credentials (VCs) become mainstream, you'll be able to export your evidence hashes directly into them. The SHA-256 hashes in your manifest are already in the right format. **Hybrid wins.**

Open Invitation:

This might be overkill. Wallets are probably coming and they're better for privacy. Tell me what sucks. Roast it. Fork it. Make it better.

Feedback > ego. — github.com/jeffreykerekes/kerekes-handshake

11. Implementation Checklist

v1.5 Compliant

- **Resume:** HTML tagged with `data-kcm` on all claim containers.
- **Text Bridge:** Every PDF in `/evidence/` has a matching `.txt` sidecar.
- **Registry:** `claims.json` maps every claim ID to at least one evidence URI.
- **Integrity Seal:** `site_manifest.json.asc` is PGP-signed and current.
- **Open Access:** CORS headers enabled for `/evidence/`.
- **Audit Portal:** `/query/` is live and serves the Forensic Manifesto.

v1.6 Compliant (superset of v1.5)

- All v1.5 requirements above.
- **Vault Resume Format:** Narrative Layer uses dense-stub presentation with `REF: [claim_id]` anchors.
- **Universal Schema:** `claims.json` uses the universal actor/claim/evidence structure where applicable.
- **External Verification:** High-stakes claims include `external_verification` links to public records.

12. Defensive License — CC BY-SA 4.0

This protocol and all associated documentation are licensed under the **Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)**.

- **Attribution:** Credit Jeffrey Kerekes as the original architect.
- **ShareAlike:** Derivatives must use the same license.
- **Purpose:** Ensures the Handshake remains a free, open utility.

PGP Fingerprint: `D39E 4ACE A4FE 3E6B 547F 58C4 6174 3446 DFA7 D48F`

Verification: `curl -s https://jeffreykerekes.com/verify/pubkey.txt`

13. The Universal Claims Standard (v1.6)

> *"Everyone is racing to put property deeds on the blockchain — but who is putting the new roof receipt in claims.json?"*

The Kerekes Handshake™ is not a resume tool. The same architecture that verifies a professional's career verifies a plumber's license, a property's condition history, a nonprofit's financials, or a politician's 30-year voting record. The protocol is **domain-agnostic**.

The Universal Circuit

```
Actor &rarr; Claim &rarr; Artifact Vault &rarr; Verification
```

- **Actor:** A person, company, product, property, or institution making a claim.
- **Claim:** The specific statement being asserted.
- **Artifact Vault:** Primary, raw evidence — PDFs, permits, datasets, recordings.
- **Verification:** A human or AI agent auditing the vault directly.

The Core Problem: The Keyword Genius Paradox

AI has made everyone a 'keyword genius.' A perfect resume, a flawless product description, and a compelling political narrative can all be generated in seconds. When everyone is optimized, the signal-to-noise ratio drops to zero. The Handshake ignores keywords. It audits provenance.

Use Cases

Actor	Claim	The Vault
Job candidate	"Saved the city $3.4M"	Budget spreadsheets, FOI rulings, press coverage
Joe the Plumber	"Licensed for gas lines"	Permits, insurance certs, inspection photos
LG Dishwasher	"Cleans 30% better"	Lab data, Energy Star ratings, service manual
Real estate	"New roof (2022)"	Paid invoice, permit, inspection sign-off
Nonprofit	"90% to the field"	PGP-signed audits, ledger snapshots
Politician	"Voted for the environment"	30-year roll-call history, donation records
Musician	"Professional pitch control"	Raw .wav stem — no auto-tune
Designer	"Expert in non-destructive editing"	.psd source file with full layer stack

Selected Use Cases in Detail

Trades & Local Services — "Joe the Plumber"

A QR code on the service van links to a `/query` portal. An AI assistant can **"interview the van"** before the technician steps into your home — verifying active insurance, license status, and whether recent jobs passed municipal inspection. Shift: from trusting a brand to verifying a system.

Real Estate & Fractional Investment

Each property acts as its own Actor with a `claims.json`. You don't buy the listing; you buy the vault. An investor in Mumbai can audit the roof receipt, permit history, and rent roll before wiring a fractional share payment — automatically, without a site visit.

Civic & Political Accountability — "The People's Audit"

Independent researchers build forensic dossiers without Super PAC budgets. A voter's AI can audit 30 years of legislative record: *"How many times did this politician vote for an environmental measure while simultaneously receiving donations from affected industries?"* The AI doesn't guess — it audits the `claims.json`.

> *The Kerekes Handshake does not replace journalism. It enhances it by providing direct access to primary artifacts.*

The Deterministic Collision

The `external_verification` field links directly to `.gov` records, court databases, and regulatory filings. If a provided PDF conflicts with the public congressional record or licensing database, the Handshake fails automatically — a **Deterministic Collision** that no amount of narrative can override. No editorial judgment required. Just the math.

The Universal Evidence Endpoint

Every product, person, property, and project should host a standardized `/.well-known/claims.json` endpoint. This is how real standards spread — not through mandates, but through adoption. A plumber hosts one on their business site. A property developer hosts one per listing. A nonprofit hosts one for their annual report. Any AI agent that understands the protocol can audit any of them.

The Ice Cream Float

You are not building the PDF reader (ice cream) or the PGP library (soda). You are engineering the system that combines them into a verifiable signal. The ingredients are open. The architecture is the invention. It is a signal, not a product.

Conclusion

The Kerekes Handshake transforms claims into verifiable systems. It started as a resume protocol and became something larger: a universal architecture for moving society from probabilistic trust to deterministic verification.

The Vault Resume is the reference implementation — one person's 30-year career, auditable by any AI in six seconds. But the same architecture applies wherever a claim is made and evidence exists to back it up: a van with a QR code, a property listing with a permit history, a politician's voting record, a nonprofit's ledger.

It is deliberately simple. It is deliberately open. It is designed to be forked, improved, and eventually superseded by something better.

Created by Jeffrey Kerekes — Systems Practitioner
jeffreykerekes.com | github.com/jeffreykerekes/kerekes-handshake

End Full Specification v1.6